Task 1: Diagnosis of the broken design

1.1

Subsystem	Label in Figure 1	Pattern it should be
Movement	Strategy	State
Routes	State	Strategy
Map	Composite	Composite (right label, wrong structure)
Place features	Decorator	Decorator (right label, done with inheritance)
Biomes	Abstract Factory	Abstract Factory (right label, no abstract factory or products)

The swapped pair is Movement and Routes. Movement is labelled Strategy and Routes is labelled State, it should be the other way round.

Strategy's intent is to have a family of interchangeable algorithms that all do the same job, so the client can pick one. Choosing a route (shortest, fastest, scenic, cheapest) is exactly that: each one answers "get me a route" in a different way, and Trip picks which to use. Trip itself does not change when the algorithm changes.

State's intent is to let an object change its behaviour when its internal state changes, so it looks like it changed class. Going from walking to flying is not a different algorithm for the same task, the traveller is now in a different condition with different rules for what it can do and where it can go, and the mode changes by rules (a transition) rather than being handed in by the client. That is State. Figure 1 puts these labels on the wrong subsystems.

1.2

Movement: State

Context: Traveller (currently holds modeCode : int instead of a state object)

State: should be an abstract TravelMode, MoveStrategy is the closest thing but is concrete

ConcreteState: missing, the modes are just the cases in the switch in doMove(mode:int)

Routes: Strategy

Context: Trip (currently holds routeType : string instead of a strategy object)

Strategy: should be an abstract RouteStrategy, RouteState is the closest thing but is concrete

ConcreteStrategy: missing, each route type is a branch of the if / else in getRoute(type:string)

Map: Composite

Component: missing, Map, Region and Location share no common base

Leaf: Location

Composite: Map (has a locations vector and a regions vector), Region is named like a composite but has no children, so it cannot nest

Client: GameManager, calling print() on the concrete classes rather than through a common interface

Place features: Decorator

Component: missing, should be the same abstract Place used by the Composite

ConcreteComponent: Location

Decorator: missing, there is no wrapper class that holds a Place

ConcreteDecorator: WeatherLocation, TollLocation, QuestLocation should be these, in Figure 1 they are subclasses of Location, and the combinations (WeatherTollLocation, TollQuestLocation, WeatherTollQuestLocation) are more subclasses

Biomes: Abstract Factory

AbstractFactory: missing, should declare createTerrain(), createNPC(), createObstacle()

ConcreteFactory: should be one per biome (DesertFactory, OceanFactory, ...), WorldBuilder is one concrete class trying to be all of them through make(biome, kind)

AbstractProduct: missing, should be Terrain, NPC and Obstacle interfaces

ConcreteProduct: DesertTerrain, OceanTerrain, DesertNPC, OceanNPC, CityNPC, ForestObstacle

Client: GameManager, which gets back void* and so has to know the concrete types anyway

1.3

Wrong pattern choice

Place features use a subclass for every combination (WeatherTollLocation, TollQuestLocation, ...) instead of a Decorator. Rule broken: favour composition over

inheritance. Consequence (maintenance): class explosion, a fourth feature needs eight new classes, and features cannot be added or removed at runtime.

Movement and Routes use a switch on an int and an if/else on a string instead of State and Strategy objects. Rule broken: Open-Closed Principle. Consequence (maintenance): every new mode or route means editing existing, tested code.

Wrong UML notation

No class or operation is italicised, so nothing is marked abstract (MoveStrategy, RouteState, Map, WorldBuilder all appear concrete). Rule broken: abstract classes and operations must be italicised. Consequence (maintenance): the reader cannot tell what to subclass and what to instantiate.

Map, Region and Location each have their own name and print() with no common Component base, and Region has no children. Rule broken: Leaf and Composite must share one Component, and the Composite must contain Components recursively. Consequence (maintenance): the client needs separate code for each class and regions cannot nest.

Broken ownership / missing virtual destructors

No base class has a virtual destructor. Rule broken: a polymorphic base class needs a virtual destructor. Consequence (runtime): deleting through a base pointer skips the derived destructor, so memory leaks.

Map is drawn with filled diamonds on its locations and regions vectors but never deletes them, the same for decorated locations and WorldBuilder products. Rule broken: a composition owner must delete its parts. Consequence (runtime): the whole map and all biome content leak.

Excessive coupling

WorldBuilder::make(biome, kind) is one giant switch that knows every product and returns void*. Rule broken: Abstract Factory needs one concrete factory per family behind an abstract interface, returning abstract products. Consequence (runtime and maintenance): no type safety, families can be mixed, and every new biome edits the one class everything depends on.

GameManager stores mode and routeType, drives move(), plan() and build(), and has makeThing() : void*, so it depends on every subsystem. Rule broken: Single Responsibility Principle. Consequence (maintenance): any change anywhere ripples into GameManager, nothing can be tested or reused on its own.

1.4

GameManager. It holds the traveller's mode and the trip's routeType as plain ints and strings, calls move(), plan() and build() itself, and makes content through makeThing() : void*. Every other class hangs off it.

The five patterns exist to put a small abstract interface between a client and a set of changing implementations, so the client only depends on the interface. Putting every decision in GameManager throws that away: it picks the mode, route, biome and feature by checking ints and strings, so it is tied to every concrete class. Adding one travel mode or biome means changing the manager, and a bug in the manager breaks all five subsystems. High coupling, low cohesion, and nothing reusable.

What should own what instead:

Traveller (State Context) owns its current TravelMode and is the only class that switches modes.

Trip (Strategy Context) owns its current RouteStrategy and has a setStrategy().

The root Region (Composite) owns and deletes its child Places.

Each PlaceDecorator owns the one Place it wraps.

Each concrete BiomeFactory decides which Terrain, NPC and Obstacle go together, GameManager only holds a BiomeFactory*.

GameManager becomes a thin driver that wires these up and runs the loop.

1.5

Figure 1 is hard to read because it is printed rotated, lines cross each other and run behind class boxes, base classes are not placed above their subclasses (WeatherTollLocation and TollQuestLocation sit level with the classes they extend, and Location has arrows both in and out, so its role is unclear), and the classes for each pattern are spread around instead of grouped. Diamonds and arrows also have no multiplicities or role names.

To improve it: draw it in portrait, group each pattern in its own labelled package with the abstract base at the top and concrete classes below so inheritance always points up, put GameManager at the top and the shared Place interface between the Composite and Decorator groups, route lines around boxes with as few crossings as possible, and add multiplicities and role names.

Task 2.2)

State:

Context - Traveller
State - TransportationMethod
ConcreteStateA - TeleportationCircle
ConcreteStateB - MerchantCaravan
ConcreteStateC - Horseback

We made the Traveller own the Transportation method to reduce the amount of memory owned by the main function to make freeing memory easier

Strategy:

Context - GameManager
Strategy - Route
ConcreteStrategyA - WarmestRoute
ConcreteStrategyB - CheapestRoute
ConcreteStrategyC - FastestRoute

Since the routefinding method can be changed on the fly, rather than storing the entire trip and generating it over, the Concrete Routes simply give the next Settlement according to the criteria.

Composite:

Tree - Place
BaseNode - Settlement
IntermediateNode - Region

We made the GameManager own the root of the world map to remove responsibility from main.

Decorator:

Component - Settlement
ConcreteComponent - City
Decorator - SettlementFeature
ConcreteDecoratorA - Food
ConcreteDecoratorB - Work
ConcreteDecoratorC - Lodging
ConcreteDecoratorD - Market

Rather than have the Place interface be the component, we made the BaseNode(Settlement) be the component so we can add interesting things for the player to experience within the cities, bringing them alive.

Abstract Factory:

AbstractFactory - BiomeFactory
ConcreteFactoryA - ForestFactory
ConcreteFactoryB - OceanFactory
AbstractProductA - Food
ConcreteProductA1 - OceanFood
ConcreteProductA2 - ForestFood
AbstractProductB - Market
ConcreteProductB1 - OceanMarket
ConcreteProductB2 - ForestMarket

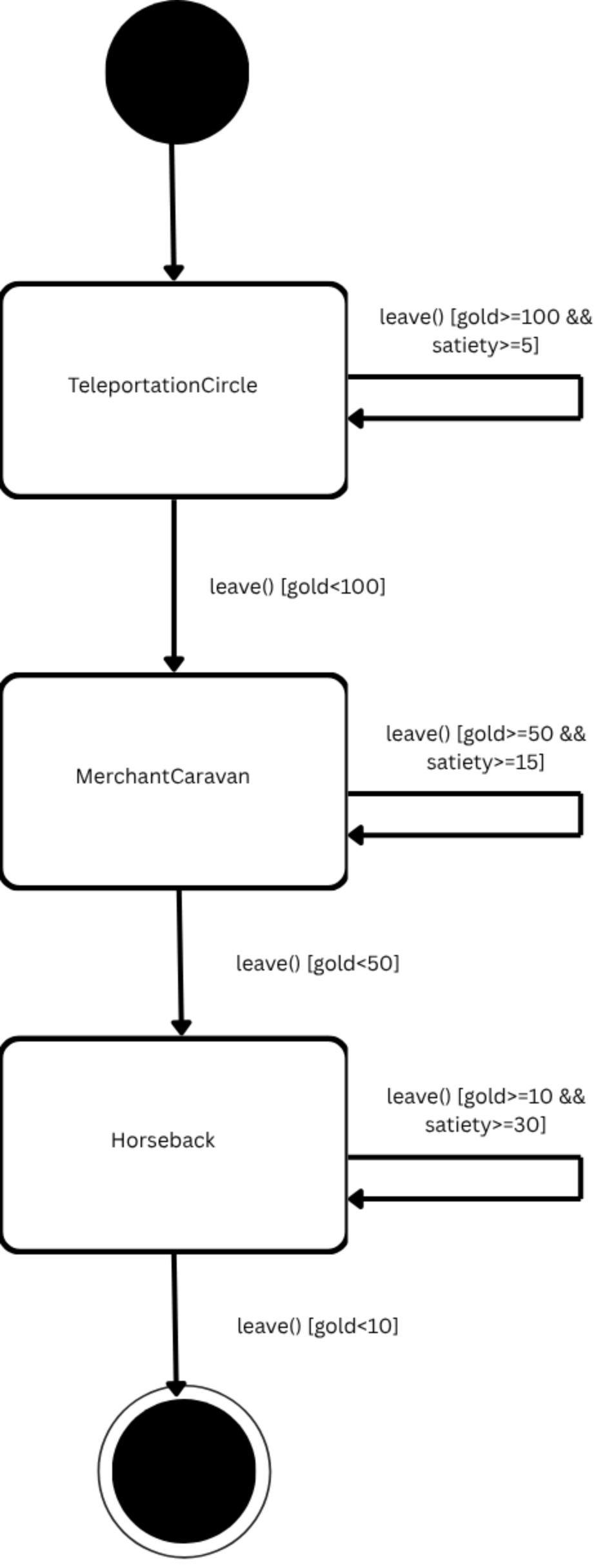
We made the abstract factory produce concrete decorators which can be added to a city.

Task 2.3)

TeleportationCircle - State
Make it inherit from transportationMethod and implement the leave function then it slots in without needing to change anything else.

OceanFood - Decorator
It inherits from food, meaning it just needs to implement eat() then it can be used like any other decorator

WarmestRoute - Strategy
Inheriting from Route, as long as it implements getNextSettlement() it can be used in place of any pre-existing Route subclass.


TeleportationCircle
leave() [gold>=100 && satiety>=5]
leave() [gold<100]
MerchantCaravan
leave() [gold>=50 && satiety>=15]
leave() [gold<50]
Horseback
leave() [gold>=10 && satiety>=30]
leave() [gold<10]

Task 4.3)

Routes are a Strategy because the Trip has one job, giving the next settlement, and there are several swappable ways to do it. Warmest, fastest and cheapest are whole algorithms for the same request. The player picks one and can change it whenever they like. Swapping the route doesn't change what the Trip is, only how the answer is worked out. That is Strategy: a family of interchangeable algorithms chosen by the client.

Movement is State because the Traveller changes its own behaviour as its condition changes. When gold drops below the cost of the current mode, the mode itself puts the next one in place. The player might pick a starting mode, but the transitions are never theirs to make. That is State: an object that alters its behaviour when its internal state changes.

The diagrams look the same because both hand work off to an abstract interface with three subclasses. The difference is who decides. In Strategy the client decides and any strategy is valid at any time, CheapestRoute doesn't even know FastestRoute exists. In State the states decide, and only certain states may follow others, TeleportationCircle knows about MerchantCaravan and creates it. Routes as State would mean a Route deciding when to turn into another Route, which makes no sense. Movement as Strategy would mean GameManager checking the gold and picking a mode, which puts the rules in the client, the very fault from Task 1.

world: Region

name = “Wayfarer”

children

children

coast: Region

name = “Jeffery’s bay”

woods: Region

name = “WoodedArea”

port: OceanCity

name = “Port”

arch: Region

name = “Shutter isle”

oakhollow:ForestCity

name = “oaky”

cityWorld: OceanCity

name = “Reefcity”

fernsPlace:ForestCity

name = “Ferny”